

```

In [1]: print("AYAAN SAIFI 015")

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

np.random.seed(42)

N = 100
x = np.linspace(0, 2 * np.pi, N)
noise = np.random.normal(loc=0, scale=0.1, size=N)
y = np.sin(x) + noise
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

plt.figure(figsize=(6, 4))
plt.scatter(x, y, color="blue")
plt.title("Feature (x) vs Target (y)")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.show()

x_mean, x_std = x_train.mean(), x_train.std()
x_train_n = (x_train - x_mean) / x_std
x_test_n = (x_test - x_mean) / x_std
x_all_n = (x - x_mean) / x_std

def make_poly_features(x_vec, degree):
    """Turn a 1-D array into [1, x, x^2, ..., x^degree] columns."""
    X = np.ones((len(x_vec), degree + 1))
    for d in range(1, degree + 1):
        X[:, d] = x_vec ** d
    return X

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def gradient_descent(X, y, lr=0.01, epochs=5000):
    m, n = X.shape
    theta = np.zeros(n)
    for _ in range(epochs):
        y_pred = X @ theta
        grad = (2 / m) * X.T @ (y_pred - y)
        theta -= lr * grad
    return theta

```

```

degrees = range(10)
coefficients, train_errors, test_errors = [], [], []

fig, axes = plt.subplots(2, 5, figsize=(20, 8))
axes = axes.ravel()

for M in degrees:
    X_train_poly = make_poly_features(x_train_n, M)
    X_test_poly = make_poly_features(x_test_n, M)

    theta = gradient_descent(X_train_poly, y_train)
    coefficients.append(theta)

    train_errors.append(mse(y_train, X_train_poly @ theta))
    test_errors.append(mse(y_test, X_test_poly @ theta))

    X_full_poly = make_poly_features(x_all_n, M)
    y_curve = X_full_poly @ theta
    order = np.argsort(x)

    axes[M].scatter(x, y, s=12)
    axes[M].plot(x[order], y_curve[order], color="red", linewidth=2)
    axes[M].set_title(f"M = {M}")

plt.tight_layout()
plt.show()

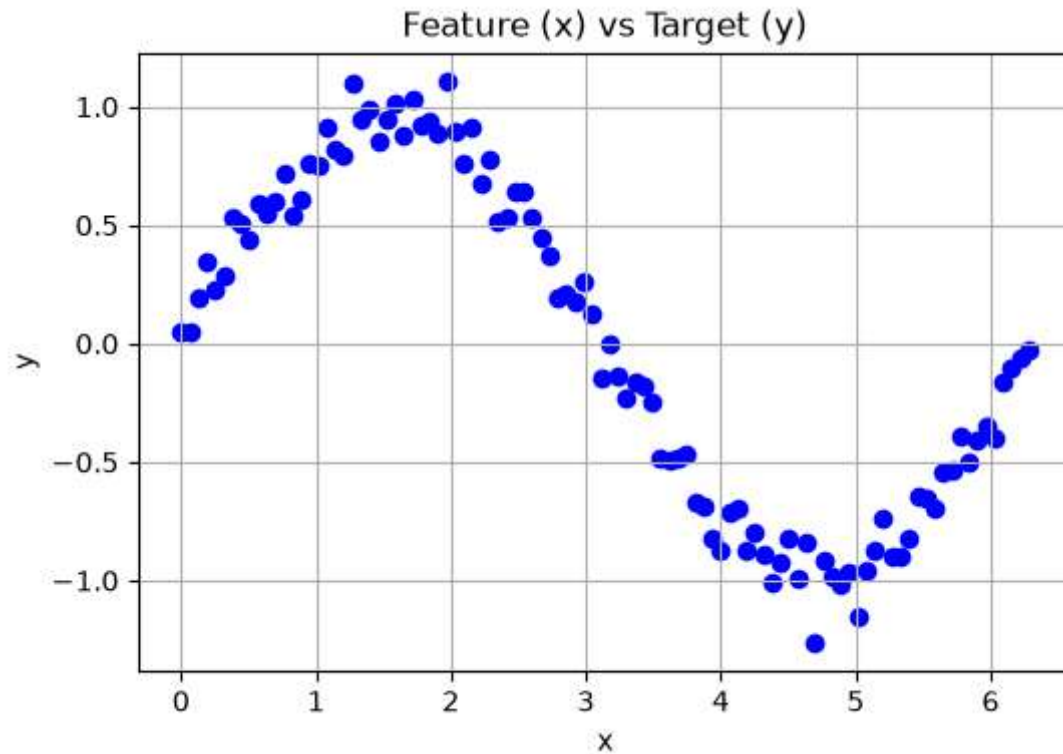
print("\nCOEFFICIENTS FOR EACH MODEL")
for M in degrees:
    print(f"Degree {M}: {coefficients[M]}")
plt.figure(figsize=(7, 5))
plt.plot(degrees, train_errors, marker="o", label="Train Error")
plt.plot(degrees, test_errors, marker="s", label="Test Error")
plt.xlabel("Polynomial Degree (M)")
plt.ylabel("MSE")
plt.title("Train vs Test Error")
plt.legend()
plt.grid(True)
plt.show()

best_M = int(np.argmin(test_errors))
worst_M = int(np.argmax(test_errors))

print("\nRESULTS")
print("-----")
print(f"Best model : degree {best_M}, test MSE = {test_errors[best_M]:.5f}")
print(f"Worst model: degree {worst_M}, test MSE = {test_errors[worst_M]:.5f}")

```

AYAAN SAIFI 015



C:\Users\Lenovo\AppData\Local\Temp\ipykernel_19988\2871337030.py:52: RuntimeWarning: overflow encountered in matmul

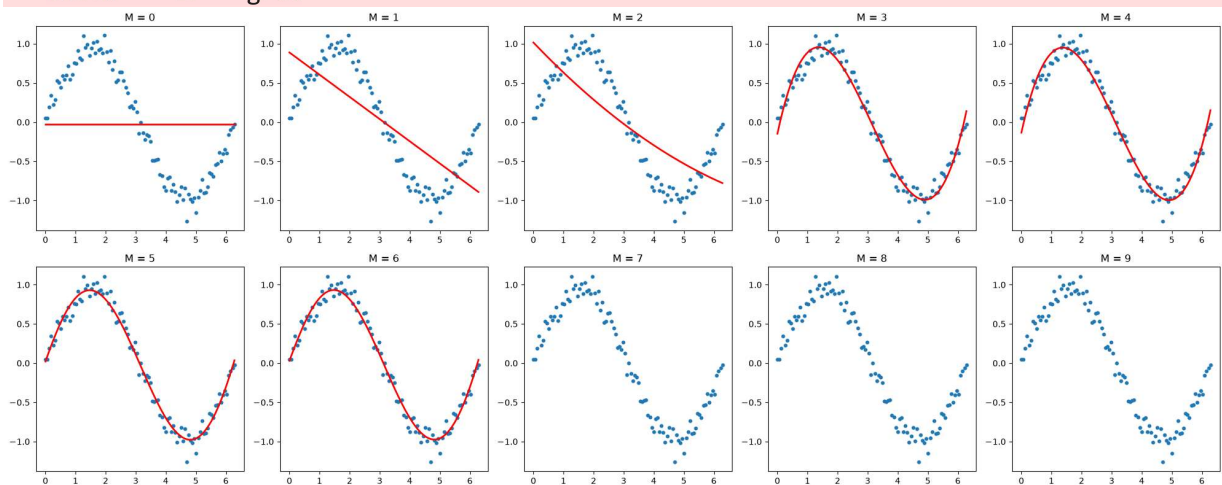
```
grad = (2 / m) * X.T @ (y_pred - y)
```

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_19988\2871337030.py:52: RuntimeWarning: invalid value encountered in matmul

```
grad = (2 / m) * X.T @ (y_pred - y)
```

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_19988\2871337030.py:53: RuntimeWarning: invalid value encountered in subtract

```
theta -= lr * grad
```



COEFFICIENTS FOR EACH MODEL

Degree 0: [-0.02332312]

Degree 1: [-0.02332312 -0.51870608]

Degree 2: [-0.08528276 -0.51735195 0.06195963]

Degree 3: [-0.08891222 -1.51067491 0.07746913 0.54358373]

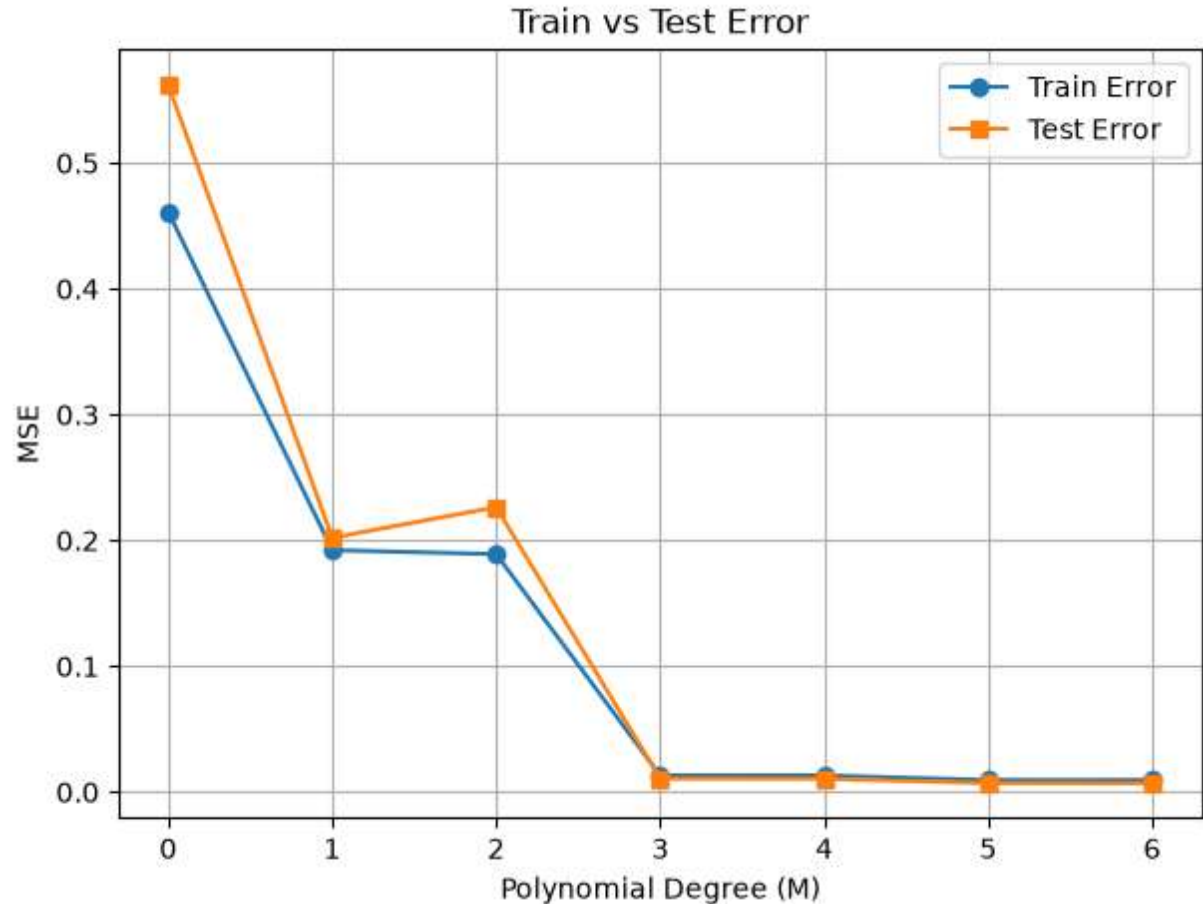
Degree 4: [-0.08431505 -1.51196595 0.06101202 0.54439876 0.00649109]

Degree 5: [-8.43947262e-02 -1.60726923e+00 7.39428568e-02 7.29851987e-01
-1.19739103e-03 -6.12136321e-02]Degree 6: [-0.08682782 -1.60648438 0.09294703 0.72836155 -0.02071155 -0.06063077
0.00485995]

Degree 7: [nan nan nan nan nan nan nan nan]

Degree 8: [nan nan nan nan nan nan nan nan nan]

Degree 9: [nan nan nan nan nan nan nan nan nan nan]



RESULTS

Best model : degree 7, test MSE = nan

Worst model: degree 7, test MSE = nan